

Quantization and Pruning-Aware Deep Recurrent Models for Real-Time Human Activity Recognition on Android

Harshit Singh

Department of Computer Science
Colorado State University
harshit.singh@colostate.edu

Trisha Ghali

Department of Computer Science
Colorado State University
trisha.ghali@colostate.edu

Abstract—Human Activity Recognition (HAR) through smartphones is a core capability for mobile health, safety monitoring, and context-aware applications. Although deep neural models such as CNN-LSTM hybrids often achieve strong accuracy on benchmark datasets, deploying them on resource-constrained phones is challenging due to limited memory, compute throughput, and energy budgets under continuous sensing.

In this project, we designed and evaluated two compact recurrent architectures for on-device HAR: (i) a lightweight CNN-LSTM model that combines temporal convolution for local motion features with stacked LSTMs for sequence modeling, and (ii) a GRU-based model as an efficiency-focused alternative. We evaluated the system on two datasets: UCI HAR and WISDM using raw inertial signals. We implemented a full TensorFlow-to-TensorFlow Lite (TFLite) deployment pipeline and applied post-training quantization (dynamic range and full integer int8), quantization-aware training (QAT) under framework constraints, and structured pruning. We additionally integrated an autoencoder-based anomaly rejection mechanism that flags out-of-distribution windows using reconstruction error thresholding.

Our evaluation reports accuracy, per-class precision/recall/F1, model size (MB), and on-device inference behavior. We highlight practical deployment constraints encountered during optimization including limited quantization support for certain recurrent and convolutional layers in QAT workflows and instability of experimental quantization formats for RNN-heavy models and show how stable int8 quantization provides an effective accuracy efficiency trade-off for mobile HAR.

Index Terms—Human Activity Recognition, LSTM, GRU, Quantization, Pruning, Embedded ML, TensorFlow Lite, Android Deployment, Anomaly Detection

I. INTRODUCTION

Human Activity Recognition (HAR) uses time-series signals from inertial sensors (e.g., accelerometers and gyroscopes) to infer a user’s physical activity. Because modern smartphones include these sensors by default, HAR can be deployed at scale without specialized hardware, enabling applications such as fitness tracking, rehabilitation monitoring, fall-risk detection, and context-aware user interfaces.

Manual feature engineering with classifiers like SVMs, decision trees, or multilayer perceptrons was the standard practice in traditional HAR pipelines. It was rigorous work to select and design features and such systems got easily affected by user variations, sensor noise, and other limitations

in the deployment area. Deep learning methods solved these problems in a way that very few if any features had to be engineered manually as they were learning the representations directly from the raw signals. CNNs captured local patterns while recurrent networks (like LSTMs/GRUs) were modeling temporal dependencies. [1], [2], [7]–[10].

Despite strong benchmark performance, deep models can be expensive on-device. Continuous sensing creates an additional energy burden: even small increases in CPU usage or latency can accumulate into noticeable battery drain. This motivates compression and optimization techniques such as post-training quantization (PTQ), quantization-aware training (QAT), and pruning, which can reduce model size and improve inference speed [2], [6]. However, in practice, deploying recurrent architectures in TFLite introduces additional constraints (kernel support, conversion limitations, and reduced-precision stability) that must be addressed for reliable mobile inference.

This work focuses on building a practical, end-to-end mobile HAR system with two compact recurrent architectures and a deployment-driven optimization pipeline. We evaluate baseline float models and multiple compressed variants (PTQ dynamic, PTQ int8, QAT where supported, pruning) and incorporate an anomaly rejection module to improve robustness when sensor windows appear unreliable. Our primary contributions are:

- Compact CNN-LSTM and GRU architectures for raw-signal HAR on mobile.
- A systematic compression pipeline: PTQ (dynamic/int8), QAT under layer-support constraints, and structured pruning.
- An autoencoder-based anomaly rejection mechanism for low-confidence or out-of-distribution windows.
- A deployment-oriented discussion of practical limitations and design choices for stable TFLite inference.

II. RELATED WORK

HAR has evolved from handcrafted-feature pipelines to deep learning models and, more recently, to deployment-aware optimization for mobile inference.

A. Deep Learning for HAR

Recurrent neural networks have been widely adopted for HAR because motion signals exhibit temporal structure. Inoue et al. [7] demonstrated high-throughput recurrent HAR, and Milenkoski et al. [1] studied real-time LSTM-based HAR on smartphones. Hybrid CNN–RNN designs are common: convolutional layers learn short-term local patterns while LSTMs/GRUs capture longer dependencies. Xia et al. [9] reported strong results with LSTM–CNN variants, and Ankita et al. [8] proposed lightweight CNN–LSTM models for smartphone data.

B. Mobile Deployment and TensorFlow Lite

Deploying HAR models on-device requires attention to latency, memory, and power. Zebin et al. [2] implemented batch-normalized deep LSTM models on smartphones and analyzed runtime behavior. Ye et al. (2019) [10] have studied real-time activity recognition, under the constraints of TFLite deployment. Open-source implementations and Android workflows [3]–[5] provide useful baselines, but typically do not evaluate a full compression pipeline across multiple model variants or analyze deployment limitations in depth.

C. Compression for Embedded Inference

Quantization and pruning are common strategies for mobile inference. PTQ reduces precision after training, while QAT trains models to be robust to quantization noise. However, recurrent architectures may be sensitive to quantization and may face toolchain constraints depending on kernel support. Surveys such as Nweke et al. [6] summarize deep HAR methods and highlight open challenges for robust real-world deployment, including compute and energy constraints.

D. Distinction from prior work

In contrast to prior work that focuses mainly on (i) accuracy-centric deep architectures or (ii) deployment without a broad compression study, we combine compact recurrent models, multiple compression techniques, and an anomaly rejection module, emphasizing stable, realistic TFLite deployment and the constraints that guide design choices.

III. DATA

This project uses two datasets for model development and evaluation: UCI HAR and WISDM. We operate on raw inertial signals rather than precomputed feature vectors to remain consistent with end-to-end deep learning and realistic mobile deployment.

A. UCI HAR Dataset

UCI HAR was collected using a Samsung Galaxy S2 at 50 Hz and includes accelerometer and gyroscope signals from 30 subjects performing six activities: *WALKING*, *WALKING_UPSTAIRS*, *WALKING_DOWNSTAIRS*, *SITTING*, *STANDING*, and *LAYING*. We use the raw inertial signal files: `body_acc_[x,y,z]_{split}.txt` and `body_gyro_[x,y,z]_{split}.txt`. The loader stacks

these into tensors of shape $(N, T, 6)$ where T is the fixed window length.

B. WISDM Dataset

The WISDM dataset provides smartphone inertial recordings for six activities (walking, jogging, upstairs, downstairs, sitting, standing). The official dataset statistics report 1,098,207 raw time-series examples with 6 attributes and no missing values, with class distribution dominated by walking and jogging (e.g., walking: 424,400 and jogging: 342,177) [11]. For this project, we use the raw time-series version to ensure architectural consistency with our UCI HAR pipeline. We include WISDM in the experiments section as an additional benchmark and we report it in parallel tables/figures to ensure a consistent comparison across datasets.

The WISDM v1.1 dataset consists of unprocessed accelerometer data from smartphones that were used in six different everyday activities: walking, jogging, going up, going down, sitting, and standing. The raw WISDM file is not like UCI HAR and it has some formatting inconsistencies, such as different delimiters, semicolons at the end of lines, and bad rows. To be able to ingest the data in a robust way, we made a custom line-wise parser that normalizes the delimiters, takes out the empty tokens, and discards the invalid samples.

A valid record is made up of a unique identifier for the user, an activity label, a timestamp, and the values of the three axes of the accelerometer. After parsing, the continuous accelerometer stream is segmented into fixed-length overlapping windows of 128 samples with a stride of 64 samples (50 percent overlap). Each window is assigned a class label using majority voting across the samples in the sampleswindow, reflecting realistic continuous sensing where transitional motion may be present.

To maintain architectural consistency with the UCI HAR pipeline, WISDM windows are represented as tensors of shape $(128, 6)$. The first three channels correspond to accelerometer measurements (ax, ay, az), while the remaining three channels are zero-padded to match the accelerometer+gyroscope interface used by the CNN–LSTM and GRU models. This design enables direct reuse of the same model architectures and deployment pipeline across both datasets.

Train–test splits are performed either by user-based separation, ensuring subject-independent evaluation, or by stratified random splitting when explicitly stated. This preparation enables fair cross-dataset comparison while preserving realistic deployment constraints.

C. Preprocessing and Windowing

Both datasets are transformed into fixed-length windows $(T, 6)$ suitable for Conv1D and recurrent layers. UCI HAR commonly uses $T = 128$ samples (2.56 seconds at 50 Hz). For WISDM, raw data are segmented into fixed windows at the dataset sampling rate (approximately 20 Hz) using a consistent window length and overlap across our experiments to enable fair comparisons.

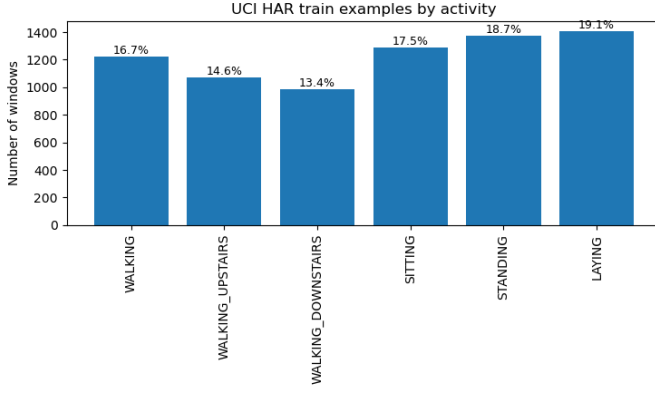


Fig. 1. UCI HAR training examples per activity class.

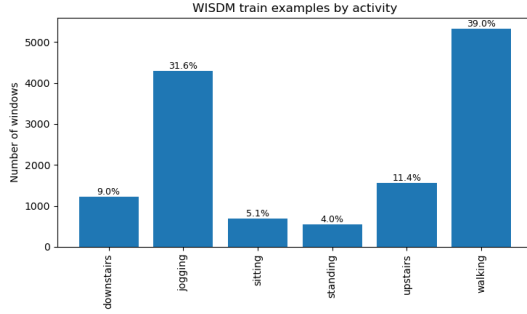


Fig. 2. WISDM raw data class distribution (based on official statistics).

D. Class Distribution and Imbalance

HAR datasets often exhibit class imbalance (e.g., more WALKING than transitional activities). This can affect macro-averaged metrics and confusion patterns (e.g., SITTING vs STANDING). We include class distribution plots for both datasets.

E. Dataset Summary

Table I summarizes key dataset properties used in our pipeline.

TABLE I
DATASET INFORMATION

Dataset	Sensors/Channels	Sampling Rate	Activities	Notes
UCI HAR	Acc+Gyro (6)	50 Hz	6	Fixed windows (commonly $T = 128$)
WISDM	Acc/Gyro pipeline (6)	~ 20 Hz	6	Raw series segmented into windows

IV. METHODS

A. System Overview

Our end-to-end pipeline consists of: (1) loading and formatting raw inertial windows $(T, 6)$, (2) training compact recurrent classifiers (CNN-LSTM and GRU), (3) compressing models for mobile inference via PTQ/QAT/pruning, (4) converting models to TFLite and running on Android, and (5) applying an autoencoder-based anomaly rejection mechanism to filter unreliable windows before reporting predictions.

Model: "model_1"		
Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 128, 6)]	0
conv1d_2 (Conv1D)	(None, 128, 64)	1984
batch_normalization_3 (Batch Normalization)	(None, 128, 64)	256
re_lu_2 (ReLU)	(None, 128, 64)	0
conv1d_3 (Conv1D)	(None, 128, 128)	24704
batch_normalization_4 (Batch Normalization)	(None, 128, 128)	512
re_lu_3 (ReLU)	(None, 128, 128)	0
dropout_1 (Dropout)	(None, 128, 128)	0
lstm_2 (LSTM)	(None, 128, 128)	131584
lstm_3 (LSTM)	(None, 64)	49408
dense_2 (Dense)	(None, 128)	8320
batch_normalization_5 (Batch Normalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 6)	774
=====		
Total params: 218,054		
Trainable params: 217,414		
Non-trainable params: 640		

Fig. 3. CNN-LSTM architecture used for baseline and compressed variants.

B. Data Loading Implementation

For UCI HAR, each inertial sensor channel is stored in a separate file and loaded as an array of shape (N, T) , with all six channels stacked into a tensor of shape $(N, T, 6)$:

$$\mathbf{X} = \text{stack}(\mathbf{a}_x, \mathbf{a}_y, \mathbf{a}_z, \mathbf{g}_x, \mathbf{g}_y, \mathbf{g}_z).$$

Labels are loaded from `y_{split}.txt` and shifted to zero-based indices for training.

In contrast, WISDM provides a single raw time-series log. Sensor readings are parsed and cleaned, segmented into fixed-length windows, and assigned activity labels using majority voting within each window. Since WISDM contains accelerometer data only, the three gyroscope channels are zero-padded so that all inputs conform to the same $(N, T, 6)$ format used by the models.

C. CNN-LSTM Model Architecture

Our CNN-LSTM classifier is designed to balance accuracy and on-device efficiency. The model takes input windows of shape $(T, 6)$ and applies:

- Two Conv1D layers (64 filters, kernel size 5; then 128 filters, kernel size 3) with batch normalization and ReLU.
- Two stacked LSTM layers (128 units with `return_sequences=True`, then 64 units).
- A dense head: Dense(128) + batch normalization + dropout + Dense(num_classes) with softmax.

The convolutional front-end extracts local motion features and reduces sensitivity to raw sensor noise and the stacked LSTMs model temporal dependencies across the window.

Model: "model_1"		
Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 128, 6)]	0
gru_2 (GRU)	(None, 128, 64)	13824
gru_3 (GRU)	(None, 32)	9408
dense_2 (Dense)	(None, 128)	4224
batch_normalization_1 (Batch Normalization)	(None, 128)	512
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 6)	774
=====		
Total params: 28,742		
Trainable params: 28,486		
Non-trainable params: 256		

Fig. 4. GRU-based architecture used for baseline and compressed variants.

D. GRU Model Architecture

As a lightweight alternative, we implement a GRU-based classifier:

- GRU(64, return_sequences=True)
- GRU(32, return_sequences=True)
- Flatten + Dense(128) + batch normalization + dropout + softmax output

The parameter count and runtime of LSTMs can be remarkably reduced by GRUs through the simplification of gating, hence, the efficiency benchmark is given under the same deployment toolchain.

E. Post-Training Quantization (PTQ)

We evaluate two PTQ configurations:

- **Dynamic range quantization:** weights quantized to int8; activations remain float at runtime.
- **Full integer int8 quantization:** both weights and activations quantized to int8 using a representative dataset for calibration.

A representative dataset generator yields windows from the training distribution to adjust activation ranges. In our experiments, full-int8 PTQ provides considerable size reduction and improves latency compared to float32 with limited accuracy loss.

F. Quantization Challenges

Deploying recurrent networks in TFLite lead to multiple toolchain constraints and failures that impacted our final design decisions:

- **INT16×8 limitations for RNNs:** Experimental mixed-precision formats such as INT16×8 did not work reliably with our RNN-heavy models. For stable deployment and interpreter allocation, we adopted full integer INT8 as the primary quantization target.
- **QAT layer-support limitations:** In our setup, QAT did not support quantizing Conv1D and recurrent layers. To ensure compatibility, we applied QAT to the Dense layers

only (“dense-only QAT”), which improves robustness of the classifier head under quantized inference but also limits the achievable file-size reduction compared to full-model quantization.

- **TensorList conversion error:** During TFLite conversion, we encountered the error that `tf.TensorListReserve` requires a static `element_shape` during the transformation pass. The suggested tool had involved turning on the Select TF Ops feature and turning off the experimental lowering of tensor-list ops. The suggestion included:

```
converter.target_spec.supported_ops =
    [TFLITE_BUILTINS, SELECT_TF_OPS]
converter._experimental_lower_tensor_list_ops = False
```
- **SELECT_TF_OPS breaking Android runtime:** Although Select TF Ops may occasionally allow conversion, we found out using SELECT_TF_OPS led to instability (runtime failures) in our Android app. Hence, we decided not to have Select TF Ops as part of the final deployment path and to use a stable SavedModel signature by creating a concrete function to explicitly specify the shapes.

G. Concrete Function Signature for Stable Export

To ensure static shapes and improve conversion stability, we exported the model with an explicit concrete function signature:

```
import tensorflow as tf
run_model = tf.function(lambda x: gru_model(x))
BATCH_SIZE = 1
STEPS = 128
INPUT_SIZE = 6
concrete_func = run_model.get_concrete_function(
    tf.TensorSpec([BATCH_SIZE, STEPS, INPUT_SIZE],
                  gru_model.inputs[0].dtype))
MODEL_DIR = f"{base_path}/GRU"
gru_model.save(MODEL_DIR, save_format="tf",
               signatures=concrete_func)
```

Empirically, we observed that exporting and converting using concrete-function signatures could change baseline float accuracy by 2–3 points compared to running the same model directly under standard Keras execution. We therefore report results in a consistent evaluation setting and explicitly document this deployment-driven behavior as a practical constraint of real-world mobile inference.

H. Pruning

We apply magnitude-based structured pruning by a polynomial decay schedule, gradually increasing sparsity between `begin_step` and `end_step`. Pruning reduces effective model complexity by removing low-magnitude weights, which can improve efficiency depending on runtime support for sparsity. We report pruning primarily as a compression/regularization technique and discuss practical implications for TFLite deployment.

I. Autoencoder-Based Anomaly Detection as a Rejection Mechanism

To improve robustness, we incorporate an autoencoder trained on normal activity windows. At inference time, reconstruction error is used to flag anomalous windows (e.g., transitional motion, unusual phone placement, or sensor noise). This module acts as a rejection mechanism, if a window’s reconstruction error exceeds a learned threshold, the pipeline can skip reporting a classification (or label as “unknown”) to reduce erroneous predictions.

Let $\mathbf{x} \in \mathbb{R}^{T \times C}$ be an input window and $\hat{\mathbf{x}}$ its autoencoder reconstruction. The anomaly score is the mean-squared reconstruction error:

$$\mathcal{E}(\mathbf{x}) = \frac{1}{TC} \sum_{t=1}^T \sum_{c=1}^C (\mathbf{x}_{t,c} - \hat{\mathbf{x}}_{t,c})^2.$$

We compute $\mathcal{E}(\mathbf{x})$ on a held-out validation set of normal windows and set the rejection threshold as a high percentile (e.g., $q = 98$ th percentile). Windows with $\mathcal{E}(\mathbf{x}) > \tau$ are flagged as anomalies.

V. EXPERIMENTS

A. Experimental Setup

Models are trained using TensorFlow 2.12 with Adam optimization. We evaluate baseline float32 models and multiple compressed variants:

- Float32 baseline (CNN–LSTM, GRU)
- PTQ dynamic range
- PTQ full integer (int8)
- Dense-only QAT
- Pruned variants
- Autoencoder-based anomaly rejection (rejection mechanism)

We report UCI HAR results and include WISDM results in the same structure, so that the final submission can present both datasets consistently.

B. Metrics

We report overall accuracy and per-class precision, recall, and F1-score. Per-class metrics are important because certain activities (e.g., SITTING vs STANDING) are commonly confused. We also report model size (MB) for exported TFLite models. For deployment characterization, we include inference latency and profiling observations (CPU behavior) using Android Studio Profiler, noting that Trepp Profiler is not applicable on Pixel 6.

C. UCI HAR: Per-Class Performance

Table II reports the per-class precision, recall, and F1-score for the UCI HAR dataset. Dynamic activities such as WALKING, WALKING_UPSTAIRS, and WALKING_DOWNSTAIRS achieve consistently high F1-scores (above 0.97), indicating that their characteristic temporal motion patterns are effectively captured by the convolutional and recurrent components of the model.

TABLE II
UCI HAR PER-CLASS PERFORMANCE

Class	Precision	Recall	F1	Support
WALKING	0.98	0.97	0.97	496
WALKING_UPSTAIRS	0.99	0.97	0.98	471
WALKING_DOWNSTAIRS	0.96	0.99	0.98	420
SITTING	0.85	0.91	0.88	491
STANDING	0.89	0.95	0.92	532
LAYING	0.95	0.82	0.88	537
Overall Accuracy	0.93 (N=2947)			
Macro Avg	0.94	0.94	0.93	2947
Weighted Avg	0.93	0.93	0.93	2947

TABLE III
WISDM PER-CLASS PERFORMANCE

Class	Precision	Recall	F1	Support
DOWNSTAIRS	0.85	0.66	0.74	339
JOGGING	0.80	0.92	0.86	1041
SITTING	0.75	0.89	0.82	244
STANDING	0.97	0.74	0.84	210
UPSTAIRS	0.65	0.76	0.70	380
WALKING	0.94	0.84	0.88	1305
Overall Accuracy	0.83 (N=3519)			
Macro Avg	0.83	0.80	0.81	3519
Weighted Avg	0.85	0.83	0.84	3519

In contrast, lower F1-scores are observed for postural activities, particularly SITTING and LAYING. These activities exhibit minimal movement and similar sensor signatures, making them more difficult to distinguish using inertial signals alone. STANDING shows higher recall than SITTING, suggesting a slight model bias toward upright postures.

Overall, the model achieves an accuracy of 93%, with macro- and weighted-average F1-scores closely aligned. This indicates stable performance across classes despite mild class imbalance and confirms that the learned representations generalize well across different activity types.

D. WISDM: Per-Class Performance

Table III presents the per-class classification performance on the WISDM dataset. Compared to UCI HAR, WISDM exhibits higher variability in sensor readings due to differences in device placement, sampling rates, and user behavior, resulting in overall lower classification accuracy.

Dynamic activities such as JOGGING and WALKING achieve higher recall and F1-scores, reflecting their distinctive motion patterns. In contrast, activities involving stair traversal (UPSTAIRS and DOWNSTAIRS) show reduced precision and recall, indicating greater overlap in their inertial signatures.

Postural activities such as SITTING and STANDING remain challenging, though STANDING benefits from higher precision, likely due to more consistent vertical acceleration patterns. Despite these challenges, the model achieves an overall accuracy of 83%, demonstrating reasonable robustness on noisier real-world data.

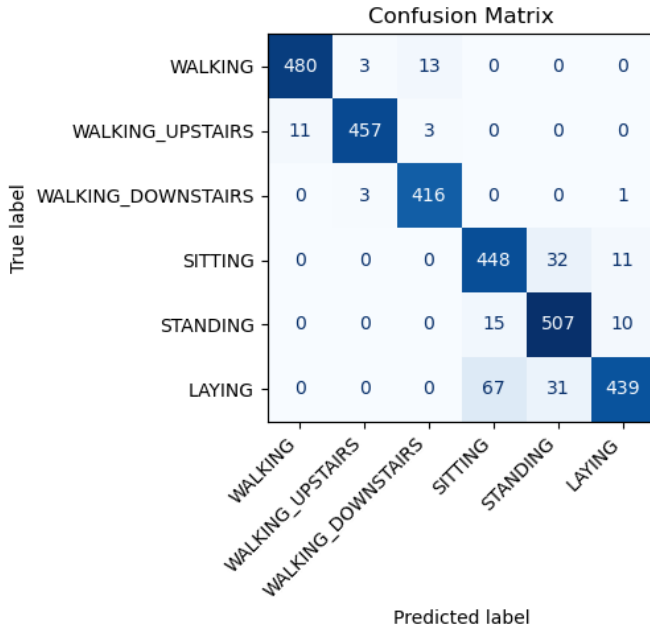


Fig. 5. Confusion matrix for CNN-LSTM float32 on UCI HAR.

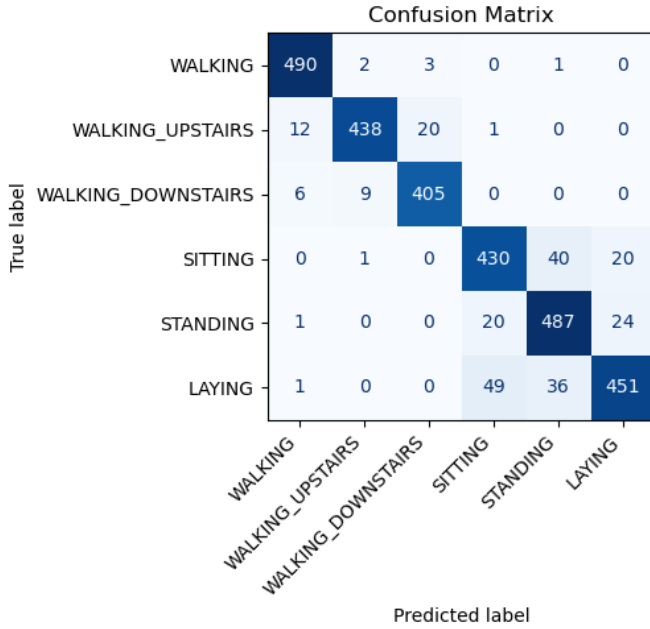


Fig. 6. Confusion matrix for GRU float32 on UCI HAR.

E. Confusion Matrix and Error Analysis

We present confusion matrices for the baseline models of both CNN-LSTM and GRU float32. Such visuals can assist in identifying and understanding the systematic confusions (for example, postural activities).

F. Accuracy-Size Trade-offs Across Optimization Stages

Table IV summarizes the main trade-off studied: accuracy versus model size across float32, PTQ, QAT, pruning, and

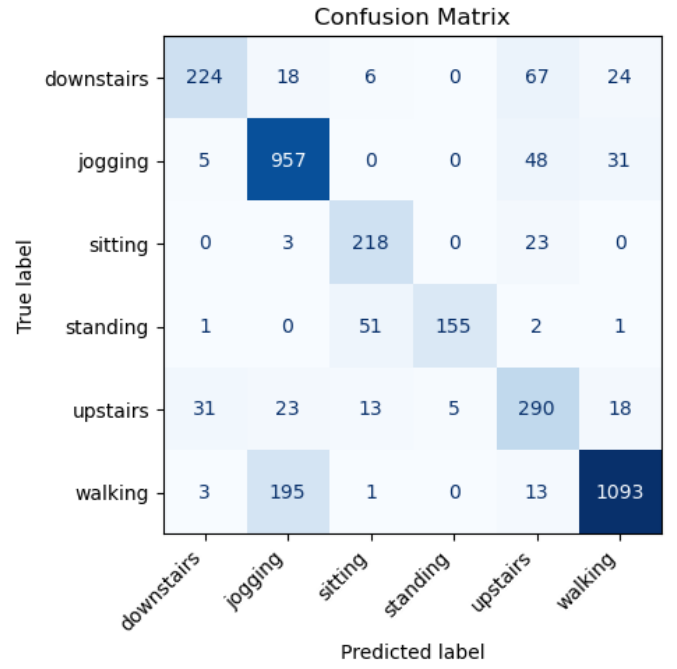


Fig. 7. Confusion matrix for CNN-LSTM float32 on WISDM.

TABLE IV
ACCURACY AND SIZE COMPARISON ACROSS MODEL VARIANTS

Dataset	Architecture	Variant	Test Acc.	TFLite Size (MB)
UCI HAR	CNN-LSTM	Float32	86.32	0.8361
UCI HAR	CNN-LSTM	PTQ Dynamic	85.85	0.22
UCI HAR	CNN-LSTM	PTQ Int8	84.86	0.224
UCI HAR	CNN-LSTM	Dense-only QAT	92.97	0.83
UCI HAR	CNN-LSTM	Pruned	82.87	1.67
UCI HAR	GRU	Float32	91.65	0.155
UCI HAR	GRU	PTQ Dynamic	91.68	0.078
UCI HAR	GRU	PTQ Int8	76.42	0.078
UCI HAR	GRU	Dense-only QAT	85.34	0.387
UCI HAR	GRU	Pruned	90.43	0.2696
WISDM	CNN-LSTM	Float32	83.20	0.83
WISDM	CNN-LSTM	PTQ Dynamic	83.12	0.22
WISDM	CNN-LSTM	PTQ Int8	82.92	0.22
WISDM	CNN-LSTM	Dense-only QAT	82.75	0.83
WISDM	GRU	Float32	84.11	0.155
WISDM	GRU	PTQ Dynamic	84.62	0.078
WISDM	GRU	PTQ Int8	82.86	0.078
WISDM	GRU	Dense-only QAT	84.68	0.83

anomaly-rejection configurations, for both CNN-LSTM and GRU, and for both datasets.

G. Training Curves

We include training/validation curves (accuracy/loss) to show convergence behavior.

H. Anomaly Rejection Evaluation

We evaluate the anomaly module as a binary detector that flags windows as anomalous if reconstruction error exceeds the threshold τ chosen from a validation percentile (e.g., $q = 98$).

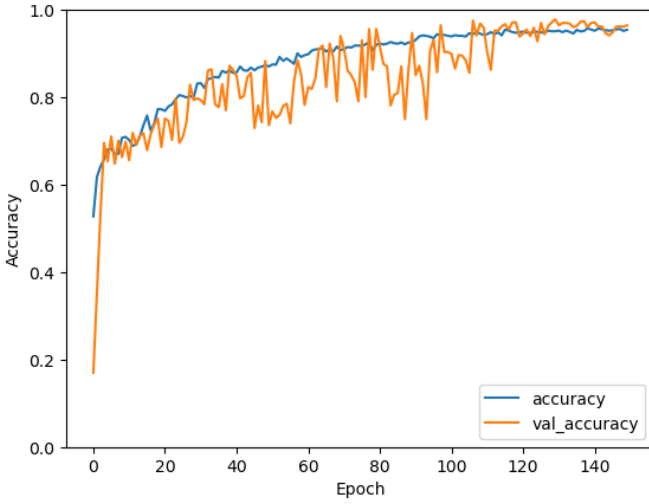


Fig. 8. Training vs validation curves for CNN-LSTM on UCI HAR.

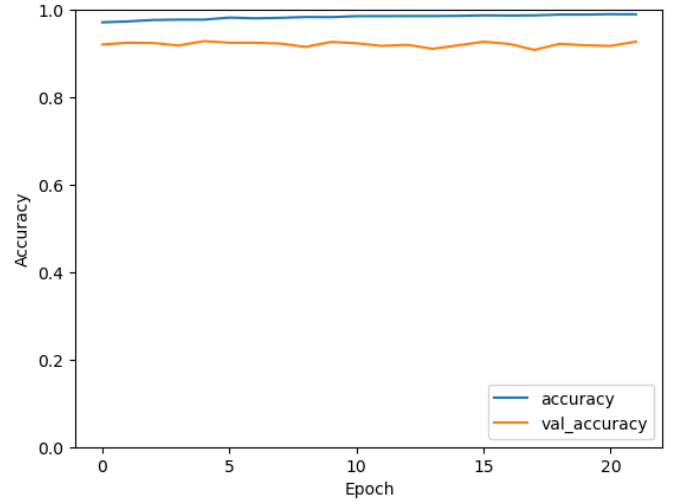


Fig. 10. Training vs validation curves for CNN-LSTM on WISDM.

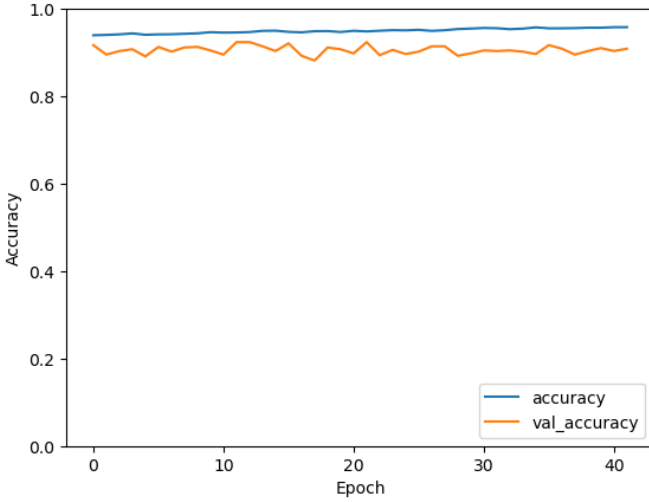


Fig. 9. Training vs validation curves for GRU on UCI HAR.

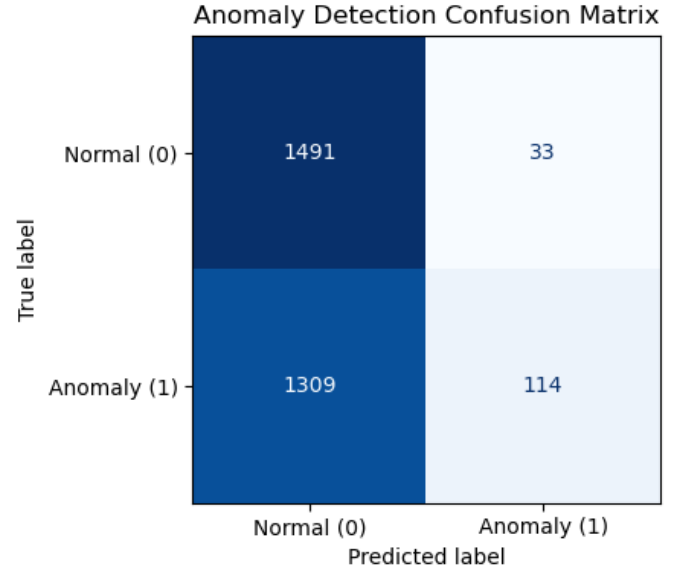


Fig. 11. Confusion matrix for anomaly detection (normal vs anomaly).

Let $y_{\text{anom}} \in \{0, 1\}$ be the anomaly ground-truth and $\hat{y}_{\text{anom}}$ the predicted anomaly label from the threshold test:

$$\hat{y}_{\text{anom}} = \mathbb{I}[\mathcal{E}(\mathbf{x}) > \tau].$$

We report precision, recall, F1, and a confusion matrix.

I. On-Device Deployment and Profiling Notes

We deploy optimized TFLite models on Android and profile inference behavior using Android Studio Profiler. Since the Pixel 6 does not use a Qualcomm chipset, Trepro Profiler is not applicable. Instead, we rely on system-level profiling and CPU behavior under continuous sensing. Include:

- average inference latency (ms) per model variant
- qualitative CPU utilization patterns (float vs int8)
- memory footprint observations

VI. CONCLUSION

This project developed and evaluated compact recurrent HAR models for real-time Android deployment using raw inertial signals from UCI HAR and WISDM. We implemented CNN-LSTM and GRU architectures and systematically evaluated deployment-oriented optimizations including post-training quantization (dynamic and full int8), dense-only QAT under toolchain constraints, and structured pruning. We further incorporated an autoencoder-based anomaly rejection mechanism based on reconstruction error thresholding to reduce unreliable predictions in ambiguous sensing conditions.

Overall, our results support the practicality of int8 quantization as a stable accuracy-efficiency trade-off for mobile HAR, while highlighting real toolchain constraints that affect which

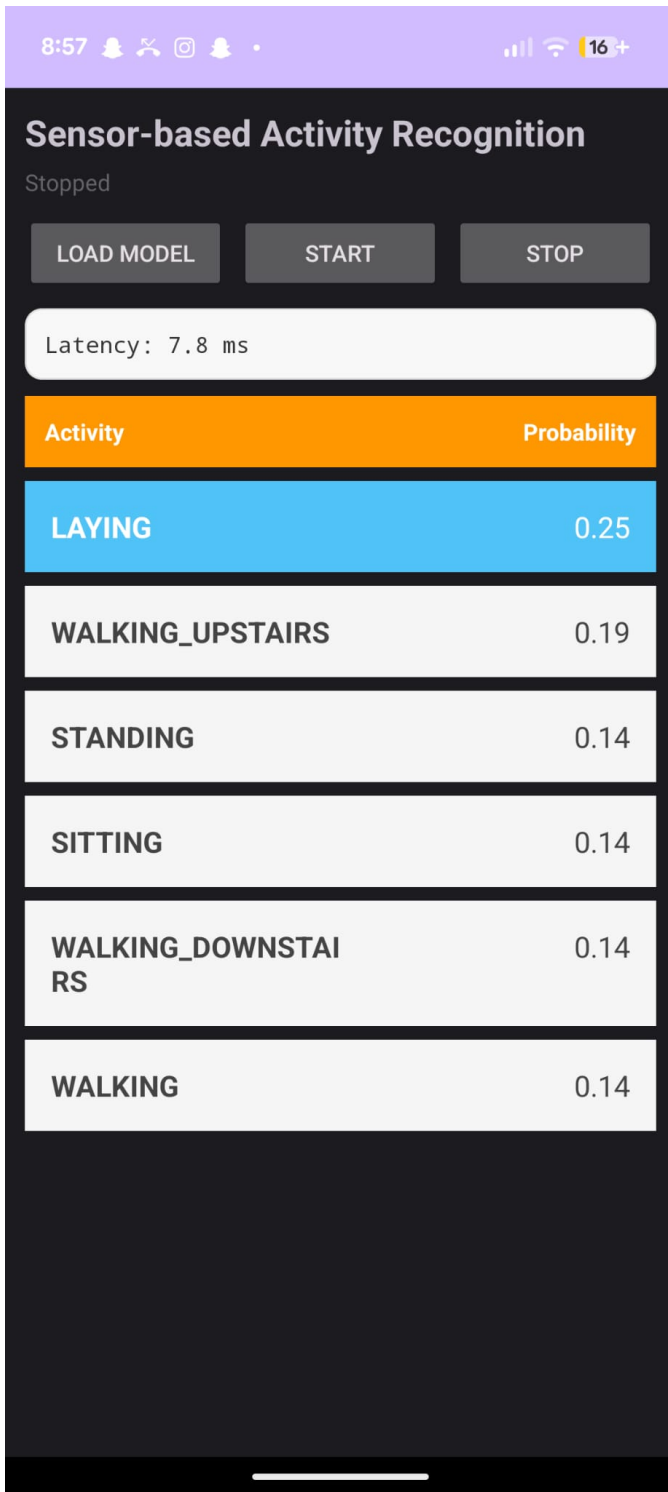


Fig. 12. Mobile UI showing real-time activity predictions.

layers can be quantized under QAT workflows. Future extensions include tighter integration of anomaly rejection with the classifier (e.g., unknown-class handling), deeper energy profiling across devices, and investigating runtime support for sparsity to better exploit pruned models.

REFERENCES

- [1] M. Milenkoski *et al.*, “Real time human activity recognition on smartphones using LSTM networks,” in *Proc. MIPRO*, IEEE, 2018.
- [2] T. Zebin *et al.*, “Implementation of a batch normalized deep LSTM recurrent network on a smartphone for human activity recognition,” in *Proc. IEEE BHI*, 2019.
- [3] G. Chevalier, “LSTMs for Human Activity Recognition,” GitHub repository, 2016. <https://github.com/guillaume-chevalier/LSTM-Human-Activity-Recognition>
- [4] D. S. Panah, “Human Activity Recognition Keras Android,” GitHub repository. <https://github.com/dspanah/Human-Activity-Recognition-Keras-Android>
- [5] V. Valkov, “TensorFlow on Android for Human Activity Recognition with LSTMs,” GitHub repository. <https://github.com/curiously/TensorFlow-on-Android-for-Human-Activity-Recognition-with-LSTMs?tab=readme-ov-file>
- [6] H. F. Nweke *et al.*, “Deep learning algorithms for human activity recognition using mobile and wearable sensor networks: State of the art and research challenges,” *Expert Systems with Applications*, vol. 105, pp. 233–261, 2018.
- [7] M. Inoue, S. Inoue, and T. Nishida, “Deep recurrent neural network for mobile human activity recognition with high throughput,” *Artificial Life and Robotics*, vol. 23, no. 2, pp. 173–185, 2018.
- [8] A. Ankita *et al.*, “An efficient and lightweight deep learning model for human activity recognition using smartphones,” *Sensors*, vol. 21, no. 11, p. 3845, 2021.
- [9] K. Xia, J. Huang, and H. Wang, “LSTM-CNN architecture for human activity recognition,” *IEEE Access*, vol. 8, pp. 56855–56866, 2020.
- [10] J. Ye *et al.*, “Deep learning-based human activity real-time recognition for pedestrian navigation,” *Sensors*, vol. 20, no. 9, p. 2574, 2020.
- [11] WISDM Lab, “WISDM Dataset Statistics,” Fordham University. <https://www.cis.fordham.edu/wisdm/dataset.php>, accessed 2025.